

Yes — **that's actually the better approach.**

If your lecturer checks individual contribution in the viva, I would **not** say:

Member 1 = AI

Member 2 = NLP/RAG

Member 3 = Security/Frontend

because then each person can become weak outside their specialty.

Instead, divide the **system into 3 functional slices**, and make **each member work across AI + NLP/IR + security + application + testing**.

For your **Adaptive Multi-Agent Cybersecurity Awareness Training System**, I'd do this:

3-Member Balanced Contribution Model

The principle is:

Every member builds one complete part of the system from end-to-end.

Each person gets:

1 agent + NLP/IR work + API integration + security + UI + testing + documentation

Member 1 — Scenario Generation & Personalization

Main feature

Adaptive Scenario Generation

This member develops the part that creates cybersecurity scenarios.

Responsibilities

AI / Agent

- Scenario Generation Agent
- LLM integration
- Scenario difficulty logic

NLP

- Extract scenario entities
- Identify threat category
- Basic text preprocessing

Information Retrieval

- Retrieve relevant cybersecurity knowledge for scenario generation

Security

- Secure scenario-generation API
- Input sanitization

Frontend

- Scenario presentation interface
- User role/difficulty selection

Database

- Store generated scenarios
- Store scenario metadata

Testing

- Test scenario quality
- Test different difficulty levels
- Test malicious/invalid inputs

Example feature

User:

Role: Finance
Difficulty: Medium
Topic: Phishing

System:

Generates a realistic finance-related phishing situation.



Member 2 — Decision Evaluation & Cybersecurity Analysis

Main feature

Evaluate the user's cybersecurity decision

This member develops the part that happens **after the user responds**.

Responsibilities

AI / Agent

- Security Analysis Agent
- Decision Evaluation Agent
- LLM reasoning

NLP

- Analyze user's written reasoning
- NER
- Text classification
- Identify security concepts

Information Retrieval

- Retrieve relevant cybersecurity guidance
- RAG for evidence-based evaluation

Security

- Secure response submission API
- Prevent malicious prompt/input injection where practical

Frontend

- Answer/decision interface
- Evaluation results screen

Database

- Store responses
- Store evaluation results
- Store identified weaknesses

Testing

- Evaluate scoring consistency
- Test correct/incorrect answers
- Test adversarial responses

Example

User answers:

"I would send the file because my manager requested it."

System identifies:

Decision: Unsafe

Possible weakness:

Authority bias

Threat:

Business Email Compromise

Score:

35/100

Member 3 — Adaptive Training & User Security

Main feature

Turn evaluation into personalized training

This member develops the system that decides:

"What should this user learn next?"

Responsibilities

AI / Agent

- Training Coach Agent

- Adaptive recommendation logic
- Next-scenario selection

NLP

- Summarize user's weaknesses
- Generate personalized feedback
- Analyze training history

Information Retrieval

- Retrieve relevant learning material
- Connect weaknesses to cybersecurity resources

Security

- Authentication
- Authorization / RBAC
- User data protection
- Encryption
- Secure user history

Frontend

- User dashboard
- Progress visualization
- Personalized training recommendations

Database

- User profiles
- Training history
- Scores
- Progress

Testing

- Authentication testing
- Access-control testing
- Recommendation testing
- Privacy testing

Example

System sees:

Phishing 82%
Social Engineering 45%
Data Protection 73%
Identity Verification 38%

Then:

Recommended next training: Identity Verification

and generates the next scenario.

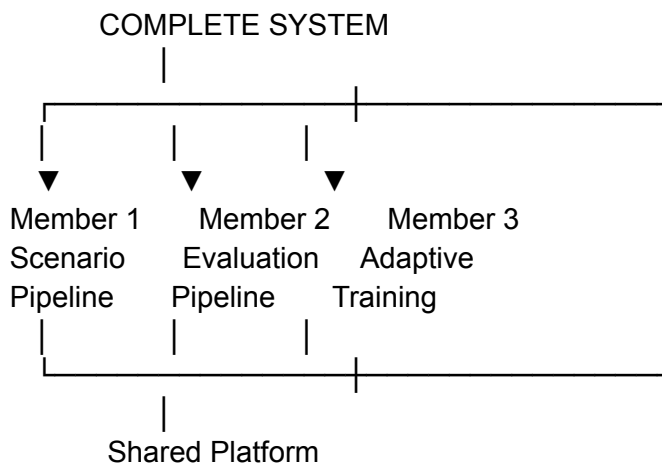
But all three members should touch the complete architecture

This is the key difference.

Your project isn't:

Member 1 → AI
Member 2 → NLP
Member 3 → Security

Instead:



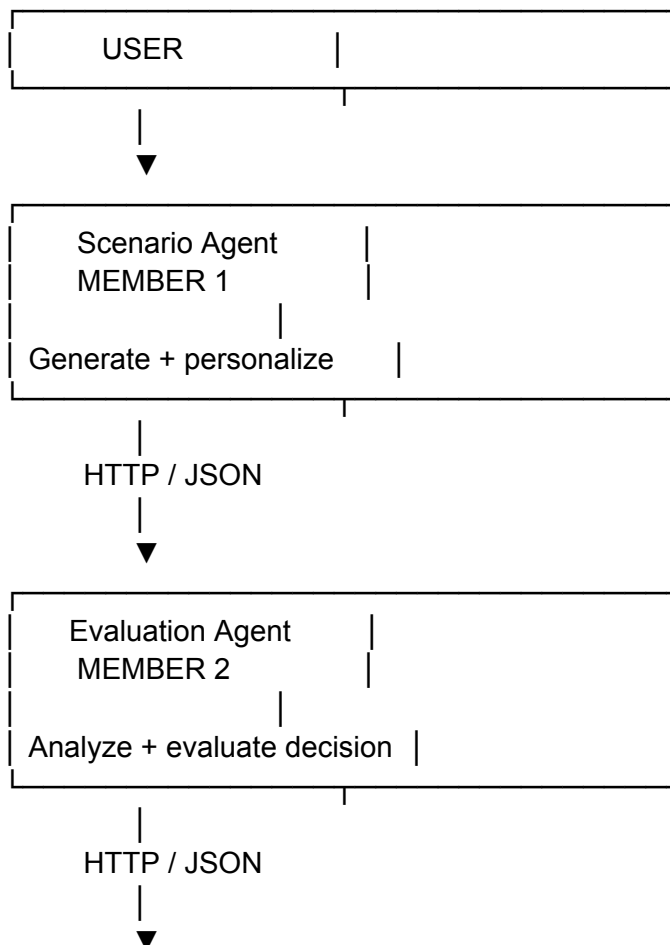
Each pipeline contains:

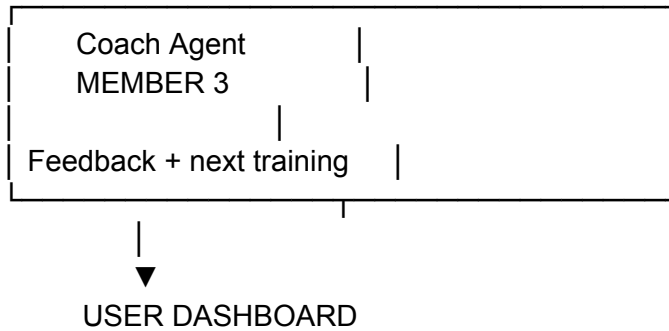
LLM
NLP
Information Retrieval
Agent Communication
API
Security
Database
Frontend
Testing

The actual agent architecture

I would now simplify your system to **3 main agents**.

That's perfect for a 3-member team.





This has a **very clean multi-agent story**.

Shared components

There should also be things that **all 3 members contribute to**.

Shared LLM layer

All members should learn how to:

- call the LLM
 - design prompts
 - validate output
 - handle failures
 - control structured JSON responses
-

Shared NLP layer

Each member implements at least one NLP function.

For example:

Member	NLP contribution
Member 1	Entity extraction
Member 2	Classification
Member 3	Summarization

Everyone therefore gets to explain NLP in the viva.

Shared Information Retrieval

Don't make one person the "RAG person."

Split the knowledge retrieval use cases:

Member 1

Retrieve information for scenario generation.

Member 2

Retrieve information for threat evaluation.

Member 3

Retrieve training/learning material.

Everyone understands the IR pipeline.

Shared security

Similarly:

Member	Security contribution
Member 1	Input sanitization
Member 2	API/agent communication security
Member 3	Authentication + authorization + data protection

But **all three should understand the entire security architecture.**

Shared communication protocol

Use:

REST API + JSON

For example:

Scenario Agent → Evaluation Agent

```
{
  "scenario_id": "SC001",
  "scenario": "...",
  "threat_type": "phishing",
  "difficulty": "medium",
  "expected_behaviors": [
    "verify_sender",
    "avoid suspicious links"
  ]
}
```

Evaluation Agent → Coach Agent

```
{
  "scenario_id": "SC001",
  "score": 42,
  "weaknesses": [
    "sender_verification",
    "urgency_bias"
  ],
  "threat_type": "phishing"
}
```

Coach Agent → User

```
{
  "feedback": "...",
  "recommended_topic": "sender verification",
  "next_difficulty": "medium-high"
}
```

This makes the **agent interaction itself** easy to demonstrate in the viva.

Individual contribution matrix

This is what I would actually put into your project plan.

Area	Member 1	Member 2	Member 3
Scenario Agent	Lead	Support	Support
Evaluation Agent	Support	Lead	Support
Coach Agent	Support	Support	Lead
LLM integration	Own module	Own module	Own module
NLP	Entity extraction	Classification	Summarization
Information Retrieval	Scenario knowledge	Threat knowledge	Training knowledge
RAG	Scenario generation	Evaluation	Feedback
Agent communication	Integration	Integration	Integration
API	Scenario API	Evaluation API	Training API
Security	Input sanitization	API security	Auth/RBAC
Database	Scenarios	Responses/results	User/history
Frontend	Scenario UI	Evaluation UI	Dashboard
Testing	Scenario tests	Evaluation tests	Security/recommendation tests
Responsible AI	Scenario fairness	Explainability	Privacy/transparency
Documentation	Scenario section	Evaluation section	Training/security section
Viva	Full system	Full system	Full system

Lead ≠ only person who knows it.

Every member should understand the other two modules at an architectural level.

How to prove individual contribution

This is extremely important.

Keep a **Contribution Log** from Day 1.

For every development task:

Date	Member	Task	Evidence
Aug 15	M1	Scenario Agent prompt design	Git commit
Aug 16	M2	NER implementation	Git commit
Aug 17	M3	Authentication	Git commit
Aug 18	M1	Scenario API	Git commit
Aug 19	M2	RAG retrieval	Git commit
Aug 20	M3	Dashboard	Git commit

Use Git/GitHub properly.

Each member should:

- Have their own Git account
- Make commits under their own account
- Create branches
- Open pull requests
- Review another member's code
- Participate in integration

That gives you **objective evidence of contribution**, rather than relying only on your report.

And rotate code reviews

This is a very good way to ensure everyone understands everything.

Member 1 develops Scenario Agent

↓

Member 2 reviews it

Member 2 develops Evaluation Agent

↓

Member 3 reviews it

Member 3 develops Coach Agent

↓

Member 1 reviews it

So everyone sees another person's implementation.

Your viva preparation should also be shared

Don't prepare:

"My part is X."

Prepare each member to answer:

Architecture

What are the three agents?

Why did you use multiple agents?

How do they communicate?

AI

Where is the LLM used?

How do you control hallucination?

NLP

What NLP techniques are implemented?

IR

Why do you need information retrieval?

Security

How is user data protected?

How do you handle malicious input?

Responsible AI

How do you ensure fairness?

How do you explain AI decisions?

Commercialization

Who would buy this?

How would you price it?

All three should be capable of answering these.

The final division I'd recommend

Member 1

Scenario Creation & Personalization

"I build the system that creates realistic cybersecurity situations."

Member 2

Decision Analysis & Evaluation

"I build the system that understands and evaluates the user's cybersecurity decision."

Member 3

Adaptive Training & User Management

"I build the system that converts the user's weaknesses into personalized training."

But underneath those three:

Every member works with AI + NLP + IR + agents + APIs + security + frontend/backend + testing.

That's the **best balance** for a 3-person university group because the contributions are distinct enough to prove ownership, but broad enough that **nobody becomes "the frontend guy" or "the AI guy."**